Task 1: Does including headings improve classification?

Jacob Low

Objective

Compare binary classification accuracy using paragraph text alone versus paragraph text plus HTML headings.

Setup

```
knitr::opts_knit$set(root.dir = normalizePath(".."))

library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.2.1     v readr     2.2.0
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.3     v tibble    3.3.1
## v lubridate 1.9.5     v tidyr     1.3.2
## v purrr     1.2.2
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(tidymodels)
```

```
## -- Attaching packages --------------------------------------- tidymodels 1.5.0 --
## v broom        1.0.13     v rsample      1.3.2
## v dials        1.4.4      v tailor       0.1.0
## v infer        1.1.0      v tune         2.1.0
## v modeldata    1.6.0      v workflows    1.3.0
## v parsnip      1.6.1      v workflowsets 1.1.1
## v recipes      1.4.0      v yardstick    1.4.0
## -- Conflicts ------------------------------------------ tidymodels_conflicts() --
## x scales::discard() masks purrr::discard()
## x dplyr::filter()   masks stats::filter()
## x recipes::fixed()  masks stringr::fixed()
## x dplyr::lag()      masks stats::lag()
## x yardstick::spec() masks readr::spec()
## x recipes::step()   masks stats::step()
```

```
library(textrecipes)
library(textrecipes)

source("../scripts/preprocessing.R")
```

```
## Loading required package: tidytext
## Loading required package: textstem
## Loading required package: koRpus.lang.en
## Loading required package: koRpus
## Loading required package: sylly
## For information on available language packages for 'koRpus', run
##
##   available.koRpus.lang()
##
## and see ?install.koRpus.lang()
##
##
## Attaching package: 'koRpus'
##
## The following object is masked from 'package:readr':
##
##     tokenize
##
## Loading required package: rvest
##
## Attaching package: 'rvest'
##
## The following object is masked from 'package:readr':
##
##     guess_encoding
##
## Loading required package: qdapRegex
##
## Attaching package: 'qdapRegex'
##
## The following object is masked from 'package:dplyr':
##
##     explain
##
## The following object is masked from 'package:ggplot2':
##
##     %+%
##
## Loading required package: stopwords
## Loading required package: tokenizers
```

```
load("../data/claims-raw.RData")
```

Extract text

```
paragraphs_only <- parse_data(claims_raw)
with_headers <- parse_data(claims_raw, include_headers = TRUE)

stopifnot(identical(paragraphs_only$.id, with_headers$.id))
```

Split data into training and test sets

```
set.seed(200)
split <- initial_split(
  paragraphs_only,
  prop = 0.8,
  strata = bclass
)

train_paragraphs <- training(split)
test_paragraphs <- testing(split)

train_headers <- with_headers %>%
  filter(.id %in% train_paragraphs$.id)

test_headers <- with_headers %>%
  filter(.id %in% test_paragraphs$.id)
```

Paragraph only

```
paragraph_recipe <- recipe(
  bclass ~ text_clean,
  data = train_paragraphs
) %>%
  step_tokenize(text_clean) %>%
  step_stopwords(text_clean) %>%
  step_tokenfilter(text_clean, max_tokens = 1000) %>%
  step_tfidf(text_clean) %>%
  step_zv(all_predictors()) %>%
  step_pca(all_numeric_predictors(), num_comp = tune())
```

logistic regression setup

```
paragraph_model <- logistic_reg() %>%
  set_engine("glm") %>%
  set_mode("classification")
paragraph_workflow <- workflow() %>%
  add_recipe(paragraph_recipe) %>%
  add_model(paragraph_model)
```

cross validation

```
set.seed(200)
cv_folds <- vfold_cv(train_paragraphs, v = 5, strata = bclass)

component_grid <- tibble(
  num_comp = c(5L, 10L, 20L, 40L)
)

paragraph_cv <- tune_grid(
  paragraph_workflow,
  resamples = cv_folds,
  grid = component_grid,
  metrics = metric_set(accuracy)
)
```

```
## > A | warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
## There were issues with some computations   A: x1There were issues with some computations   A: x3There
```

```
collect_metrics(paragraph_cv)
```

```
## # A tibble: 4 x 7
##   num_comp .metric  .estimator  mean     n std_err .config
##      <int> <chr>    <chr>      <dbl> <int>   <dbl> <chr>
## 1        5 accuracy binary     0.551     5 0.00967 pre1_mod0_post0
## 2       10 accuracy binary     0.640     5 0.0173  pre2_mod0_post0
## 3       20 accuracy binary     0.698     5 0.0117  pre3_mod0_post0
## 4       40 accuracy binary     0.739     5 0.0148  pre4_mod0_post0
```

Results of this show that 40 componenets has the greatest accuracy. We run with more components to see where improvement levels off.

```
component_grid <- tibble(
  num_comp = c(40L, 60L, 80L, 100L)
)

paragraph_cv <- tune_grid(
  paragraph_workflow,
  resamples = cv_folds,
  grid = component_grid,
  metrics = metric_set(accuracy)
)
```

```
## > A | warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
## There were issues with some computations   A: x1There were issues with some computations   A: x3There
```

```
collect_metrics(paragraph_cv)
```

```
## # A tibble: 4 x 7
##   num_comp .metric  .estimator  mean     n std_err .config
##      <int> <chr>    <chr>      <dbl> <int>   <dbl> <chr>
## 1       40 accuracy binary     0.739     5 0.0148  pre1_mod0_post0
## 2       60 accuracy binary     0.764     5 0.00855 pre2_mod0_post0
## 3       80 accuracy binary     0.769     5 0.00588 pre3_mod0_post0
## 4      100 accuracy binary     0.763     5 0.00465 pre4_mod0_post0
```

80 components seems to have performed the best among the tested values.

```
best_paragraph <- select_best(
  paragraph_cv,
  metric = "accuracy"
)

best_paragraph
```

```
## # A tibble: 1 x 2
##   num_comp .config
##      <int> <chr>
## 1       80 pre3_mod0_post0
```

```
final_paragraph_workflow <- finalize_workflow(
  paragraph_workflow,
  best_paragraph
)

paragraph_fit <- fit(
  final_paragraph_workflow,
  data = train_paragraphs
)
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

including header

```
header_recipe <- recipe(
  bclass ~ text_clean,
  data = train_headers
) %>%
  step_tokenize(text_clean) %>%
  step_stopwords(text_clean) %>%
  step_tokenfilter(text_clean, max_tokens = 1000) %>%
  step_tfidf(text_clean) %>%
  step_zv(all_predictors()) %>%
  step_pca(all_numeric_predictors(), num_comp = tune())
```

```
header_workflow <- workflow() %>%
  add_recipe(header_recipe) %>%
  add_model(paragraph_model)
```

assigning same cv splits as training

```
header_splits <- purrr::map(cv_folds$splits, function(s) {

  train_rows <- match(
    analysis(s)$.id,
    train_headers$.id
  )

  validation_rows <- match(
    assessment(s)$.id,
    train_headers$.id
  )

  stopifnot(!anyNA(train_rows), !anyNA(validation_rows))

  make_splits(
    list(
      analysis = train_rows,
      assessment = validation_rows
    ),
    data = train_headers
  )
})

header_folds <- manual_rset(
  splits = header_splits,
  ids = cv_folds$id
)
```

compontent testing for headers to find the best tuned model for headers

```
header_grid <- tibble(
  num_comp = c(5L, 10L, 20L, 40L, 60L, 80L, 100L)
)

set.seed(200)

header_cv <- tune_grid(
  header_workflow,
  resamples = header_folds,
  grid = header_grid,
  metrics = metric_set(accuracy)
)
```

```
## > A | warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
## There were issues with some computations   A: x1There were issues with some computations   A: x3There
```

```
collect_metrics(header_cv)
```

```
## # A tibble: 7 x 7
##   num_comp .metric  .estimator  mean     n std_err .config
##      <int> <chr>    <chr>      <dbl> <int>   <dbl> <chr>
## 1        5 accuracy binary     0.621     5 0.00808 pre1_mod0_post0
## 2       10 accuracy binary     0.649     5 0.0191  pre2_mod0_post0
## 3       20 accuracy binary     0.729     5 0.00496 pre3_mod0_post0
## 4       40 accuracy binary     0.752     5 0.0128  pre4_mod0_post0
## 5       60 accuracy binary     0.762     5 0.00808 pre5_mod0_post0
## 6       80 accuracy binary     0.766     5 0.00915 pre6_mod0_post0
## 7      100 accuracy binary     0.764     5 0.0135  pre7_mod0_post0
```

Our results show that the best tuned model for headers uses 80 components.

```
best_header <- select_best(header_cv, metric = "accuracy")

header_fit <- header_workflow %>%
  finalize_workflow(best_header) %>%
  fit(data = train_headers)
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

evaluating both models

```
paragraph_test_predictions <- predict(
  paragraph_fit,
  new_data = test_paragraphs
) %>%
  bind_cols(test_paragraphs %>% select(.id, bclass))

header_test_predictions <- predict(
  header_fit,
  new_data = test_headers
) %>%
  bind_cols(test_headers %>% select(.id, bclass))
  bind_rows(
  accuracy(
    paragraph_test_predictions,
    truth = bclass,
    estimate = .pred_class
  ) %>% mutate(version = "Paragraphs only"),

  accuracy(
    header_test_predictions,
    truth = bclass,
    estimate = .pred_class
  ) %>% mutate(version = "Paragraphs + headings")
) %>%
  select(version, .metric, .estimate)
```

```
## # A tibble: 2 x 3
##   version               .metric  .estimate
##   <chr>                 <chr>        <dbl>
## 1 Paragraphs only       accuracy     0.774
## 2 Paragraphs + headings accuracy     0.760
```